

Programming for Bioinformatics | BIOL 7200

Exercise 12

Submission specifications

1. Your module **does** need to be generalizable to any SAM input files.
2. **This week you may use NumPy and Biotite in addition to Python standard library modules.** The allowed use of non-standard library packages will be described below.
3. Your submission should consist of a magop.py script as well as a magnumopus module file or the contents of a package directory including a __init__.py file as in previous assignments.

Background

Assignment goal

Welcome to the final assignment of the semester! This week you will be combining the various components of what we have been calling your magnum opus. Once combined, your magnum opus will really justify that name. Each of the previous assignments has been designed to introduce or reinforce a new concept. This week we are not so much reinforcing a coding concept, but instead getting experience combining various components into a large and complex software package.

Over the course of the last few assignments there may have been points where you wondered what was the point in some of the things we were doing. For example, why bother using modules when everything could just go in a single script - then you wouldn't have to manage multiple files. Indeed, you have seen this semester through course materials, assignments, and code review sessions that you can often achieve the same things in several different ways. However, it's not always clear which is the "best" way.

I have often alluded to some coding constructs providing organizational benefits. For example, by organizing related functions and classes into modules, you know which file to go to if you want to edit those things. You are also able to control which things are exposed when the module is imported. Some functions are only needed within the module and there may only be one main function that you actually wrote to be used outside the module (e.g., you might want to use an isPCR function outside the isPCR module, but not all the functions used in intermediate steps of isPCR). My claims that it is somehow good to organize your code in ways which are often more complex or laborious to set up might not have been very convincing. The goal of this assignment is for you to see for yourself whether they are valuable.

What this assignment is intended to do is to give you an opportunity to experience the consequences of design choices we have made over the last few weeks. As you are piecing together your magnum opus, take some time to think critically about how the components are structured and whether each part is easy to use. Maybe experiment with refactoring components and see if the design of each part can make the combination of components simpler.

Some of learning to do software development is learning to see the future. As you gain more experience putting together complicated software, you get a feel for what the downstream consequences are of different design decisions. Often, when starting out on a new project, it is worth the effort to take a little bit more time

to write something in a more complex form (e.g., in a class or module) if it makes your job easier later on. The goal of this assignment is for you to actually experience those downstream consequences.

Inferring phylogenetic relationships

This section provides some optional background on what your magnum opus will be useful for after completing this assignment. You can skip to the section "Phylogenetic inference in your magnum opus" if you just want to read details that are directly related to the assignment.

If you are unfamiliar with phylogenetics take a look at [this review](#) (better description) or [this review](#) (better figures) for an overview of some key concepts and approaches.

Phylogenetics is the study of evolutionary relationships among biological entities. Most often, humans have not been around to witness (or at least not paying close attention and writing down) the evolutionary history of the forms of life around us. Therefore, in order to learn about the evolutionary relationships among organisms, genes, or any other biological unit, we must infer those relationships using other data. Collectively, methods of inferring evolutionary relationships from data are called "phylogenetic inference".

A basic assumption underpinning all phylogenetic inference methods is twofold:

1. All life arose from a single (or a small number of) common ancestor(s).
2. organisms accumulate changes over time and diverge from one another.

Inferring a phylogenetic tree is a way of trying to come up with a hypothesis of the evolutionary history of a set of organisms; it attempts to draw out when each pair of organisms diverged from one another (i.e., when was their last common ancestor). We can try to infer how long ago two organisms diverged by taking advantage of the second point above: "organisms accumulate changes over time and diverge from one another." If the accumulation of changes happens at a reasonably constant rate in all organisms then pairs of organisms that are less different will tend to also be the ones who diverged more recently. Conversely, organisms that are more different will be the ones who diverged longer ago. For example, imagine we have three organisms, A, B, and C. If A and B are more similar to one another than they are to C, then we might infer that A and B share a more recent last common ancestor than do all three organisms. Here, "last common ancestor" is the name given to a hypothetical individual from whom a set of individuals descend. For example, the last common ancestors of a person and their first cousin is their grandparents.

So, phylogenetic inference is as simple as assessing the relative similarity among a set of things. The difficult part then is two fold: for which features should you assess the similarity, and how should you quantify the similarity? These are the difficult questions in phylogenetic inference.

Until very recently, scientists were limited in the characters (i.e., comparable properties) they could use in phylogenetic inference to mostly use physiological characteristics or other phenotypic traits. For example, how many eyes does an organism have, how many limbs does it have, etc. This system worked perfectly fine for a lot of cases, particularly closely related organisms (you have likely heard of Darwin's finches, which were characterized using things like beak size and shape).

While we have learned huge amounts about life through such analyses, the use of physiological traits has weaknesses. Specifically, in order for a trait to be informative with regards to phylogeny it must be present in all organisms being considered, and it must be homologous, i.e., have changed over time from an ancestral state which was present in the last common ancestor of all the organisms being considered. The requirement that all individuals being considered have a trait makes it difficult to compare distantly related organisms

(how many comparable traits can you imagine between yourself and a slug vs yourself and a cat?).

Additionally, it is more difficult to be confident that traits in distantly related organisms are homologous. For example, bats, birds, and butterflies all have wings, but their wings are not homologous; each evolved their wings independently through convergent evolution. The lack of informative traits and issues like convergent evolution reduce the power of phenotypic analyses to resolve phylogenetic relationships over large evolutionary distances.

In recent years, DNA sequencing has become increasingly affordable and accessible. DNA sequence-based phylogenetic inference offers large advantages over phenotypic trait-based analysis. One of the largest advantages is the sheer quantity of information provided by DNA sequencing. Each base in a DNA sequence can provide information about the evolutionary history of a set of organisms.

Over the last few weeks, these assignment background sections have described how 16S sequences have become commonly used for phylogenetic inference of prokaryotic organisms (*Bacteria* and *Archaea*). This week, you are actually going to use 16S sequences to perform phylogenetic inference. To do so, we need to cover the steps required to perform phylogenetic inference.

Just like for phenotypic traits, there are some core requirements that dictate which sequences we can use for phylogenetic inference. Those are that the sequence must be present in all organisms being considered, and that differences in the sequence represent changes that have occurred over the evolutionary history of the organism. This last point is especially tricky for bacteria as many bacteria undergo frequent horizontal gene transfer (they exchange DNA with other organisms to add to or replace DNA they already had - the DNA is not transferred from parent to offspring, but between nearby cells which may or may not be the same species). The 16S gene is an ideal sequence for phylogenetic inference because the small subunit rRNA it encodes is essential for life, which means that it is reliably present (although eukaryotes have a different version of this gene - 18S). Secondly, while horizontal gene transfer of ribosomal genes like 16S has been observed, it seems to be rare. In addition, many organisms genomes contain multiple copies of the 16S rRNA gene, so if a single copy is horizontally transferred the remaining copies may still provide a consensus of what the original gene copy is. Differences in 16S sequences are therefore more reliably (than other genes) representative of changes that have happened over the evolutionary history of organisms.

Inferring a tree vs rooting a tree

For this assignment you will be generating unrooted trees. This section describes what it means for a tree to be unrooted so that you understand what your output data are, but you don't need any understanding of tree rooting for this assignment.

Perhaps the most difficult part of inferring phylogenetic relationships among organisms is not inferring a tree; it is rooting that tree. I expect that you will have seen rooted trees in most (or all) of the cases that you have seen trees. However, in many cases trees being shown as rooted are either approximately rooted (e.g., in the middle - called midpoint rooted) or not rooted at all but still plotted as if they were rooted because of the default settings of a visualization software. Rooting a tree requires more information than just the relationships among the organisms in your tree. You need some way of knowing where to put the hypothetical last common ancestor of your organisms.

But what does it mean for a tree to be unrooted? Imagine for example a tree in which the leaves are human, chimpanzee, goldfish, rabbit, and *Escherichia coli*. I've made up some branch lengths and placed those organisms into what I think is a sensible enough hypothesis of a tree:

```
((human:1,chimpanzee:1):2,(rabbit:2,(goldfish:2,Escherichia_coli:10):1):1);
```

If you give that tree to a visualization tool such as [iTOL](#) (which you definitely should) then you will see the tree represented something like the below.

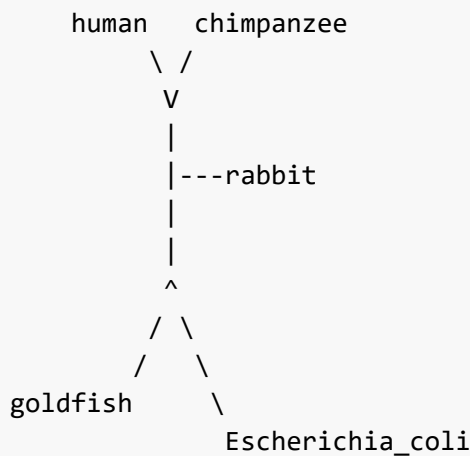
```

      +---- human
+-----|
|       +---- chimpanzee
|
|       +----- rabbit
+-----|
|       +----- goldfish
+-----|
|       +----- Escherichia_coli
```

The tree above (and what you will see in iTOL) is a rooted representation of a tree. The far left of the tree is where all the branches are coming from and each branch is some distance from the left-most point. In this representation, the left-most point is said to be the root. You might notice that this representation looks like a line graph or some other kind of plot and that there must therefore be an X and a Y axis. In a rooted tree visualization that is correct, the X axis generally represents time or mutations since the root (the root is at $X=0$), while the Y axis has no meaning and vertical lines are just longer or shorter to prevent the horizontal lines from overlapping.

In a phylogenetic tree the root represents the last common ancestor of all the things in the tree. You can read the tree from left to right to see a hypothesis of the history of the organisms in the tree. The issue with the example tree above is if you try to read the tree that way you will quickly see that the life history of the organisms in the tree is not correct. goldfish and *Escherichia coli* do not have a common ancestor that is more recent than that of rabbits and goldfish. When the history doesn't match the tree that means the tree is rooted in the wrong place. In fact, this tree has not been rooted. The branch lengths represent the differences between each pair of organisms, but the tree has not been modified to place a root so that it can be interpreted as a rooted tree.

A more accurate representation of that tree can be seen below. If you would rather view the tree in iTOL then within the "Control Panel" that is on the top right of the window click the "Basic" tab and then click "Unrooted". You should see the tree turn something that looks like a stick man or the below text representation.



An unrooted tree doesn't have an obvious X or Y axis. In fact, this tree can be flipped or rotated without changing its meaning. In unrooted trees the branches are a one-dimensional distance between points, but how each line is arranged in two-dimensional space is irrelevant (the X and Y axes are not meaningful). That means that it doesn't matter how close together two leaves are on the screen, the way to tell how distant they are is to follow the branches from one to the other and add up the branch lengths.

In the unrooted tree there is no obvious way to infer which point in the tree was the ancestor of all the organisms shown. The tree I provided you above doesn't have information about where the ancestor is (i.e., it's unrooted) and so this is the correct way to represent and think about that tree. Hopefully you can see that the relationships among the organisms in the tree make more sense in this representation - we don't know which ancestors came first, but we can see how the organisms are related.

It is possible to root an unrooted tree. There are two broad approaches: based on data in the tree or using additional data. An example of an rooting based on the tree is midpoint rooting. Midpoint rooting relies on the premise that organisms are changing over time and assumes that the change is pretty much the same speed in all the organisms in the tree. If that's true, then the middle point between the two most distant organisms is probably the last common ancestor. Midpoint rooting therefore just puts the root in the middle. That would actually work quite well for our tree and for the trees you will be making in this assignment so you can use that approach to view your trees with a sensible rooting. If you have more data about the organisms in your tree you might choose organism one that is different from all the others and call that organism the "outgroup" (e.g., E. coli in our tree is a bacterium while the rest are eukaryotes). You can then put the root somewhere on the branch between the outgroup and the rest of your tree.

Midpoint rooting is easiest and will work fine for you to look at and assess your trees so we'll stick with that. To midpoint root a tree you can use the "Advanced" tab on iTOL and click "Midpoint Root" at the bottom of the control panel. You can also do it using a command line software like [gotree](#) (one of my favourite phylogenetics tools!).

e.g.,

```

$ gotree reroot midpoint < <(echo "((human:1,chimpanzee:1):2,(rabbit:2,
(goldfish:2,Escherichia_coli:10):1):1);")
(Escherichia_coli:7.5,((rabbit:2,(human:1,chimpanzee:1):3):1,goldfish:2):2.5);
  
```

Note that the branch lengths are modified when the root is moved. Rerooting involves changing the structure of the tree and any branches attached to nodes that are moved end up modified.

For looking at the trees you will be making for this assignment I would recommend you either use iTOL (for a more polished visualization) or `gotree` (because visualizations made of ASCII characters are just cool!)

`gotree` can draw a tree in your terminal for you (though note it does so in a rooted form) and accepts piped stdin and stdout so you can chain together a rerooting and drawing command (Note I shortened the branches with `-w` so it would fit better in the page.):

```
$ gotree reroot midpoint < <(echo "((human:1,chimpanzee:1):2,(rabbit:2,
(goldfish:2,Escherichia_coli:10):1):1);") | gotree draw text -w 80
+----- Escherichia_coli
|
|               +----- rabbit
|               |
|               |               +----- human
+-----|-----+-----|-----
|               |               |
|               |               +----- chimpanzee
|               |
+-----|----- goldfish
```

Midpoint rooting will often put the root somewhere that is pretty reasonable. The branch lengths coming out of the root might not be exactly right, but for the purpose of a qualitative representation of which organisms had more or less recent common ancestors it works well.

Phylogenetic inference in your magnum opus

Currently, your magnum opus is actually a few separate things. This week we are going to combine them into a single application. The purpose of this application will be to extract 16S sequences from different data types (reads and assemblies). The data produced by your application can then be used for phylogenetic inference to classify unknown prokaryotic organisms by placing them in a phylogenetic tree based on an analysis of their 16S rRNA gene sequence.

In order to infer a phylogenetic tree based on 16S sequences we need to do three things:

1. identify sequences that are present in all the organisms being considered
2. compare the sequences to assess which are more different and which more similar among them
3. use that comparison to infer relationships among the sequences

There are a few ways we can go through that process, depending on which phylogenetic inference method we wish to use (see the reviews linked at the start of the section above). Sadly, the semester is almost over so we don't have time to implement a phylogenetic inference method ourselves (however, you can do so if you wish). Instead, we are going to prepare our data and then use tools created by others to do make the tree. What we are going to do ourselves is just the first and second step of the three steps listed above: identify sequences that are present in all the organisms being considered and compare them.

It is those two tasks that we have been the focus of the last few assignments. We have now written Python code to extract 16S sequences from two different forms of input data: reads and assemblies. We have also written code to compare sequences (Needleman-Wunsch) However, we still have a little bit of work to do to

unify the components. The sequences produced by your isPCR and reads-based workflows are not the same. Specifically, your reads-based approach currently extracts roughly the whole 16S gene, while your assembly workflow extracts just a small portion (the V4 region). In order to compare the sequences we generate from assemblies with those we generate from reads we need the sequences to be equivalent. i.e., we either need to extract the whole 16S gene from assemblies or we need to extract just the V4 region from reads.

The amplicon from the 515F and 806R primers we are using is the V4 hypervariable region of the 16S gene. Using this region alone [may not provide the best resolution of differences compared to using the whole gene](#). However, we have been using it in these assignments as it works well enough for course phylogenetic comparisons and produces a manageable amplicon. Furthermore, the primers we are using are able to produce an amplicon for highly diverse organisms. Primers that amplify the whole gene do not have the ability to amplify organisms as diverse as the V4 primers we are using. Therefore, we will be able to classify more diverse organisms if we use the small V4 region we used for our original isPCR assignment. (You should certainly experiment with other primers as well to see how they perform if you are interested - you now have code to explore those kinds of things!)

For this assignment we will extract the V4 region from our mapping data. To do so you should combine the mapping and isPCR approach. Specifically, map the reads to a 16S reference and then use isPCR and the 515F and 806R primers to amplify the V4 region. I have provided you with the subsequence of each of last week's references that corresponds to the V4 amplicon plus primer annealing sites as well as an extra 5 bases on each end. The extra 5 bases are to improve our mapping results. When mapping reads to a reference with lots of mismatches, if the first or last few bases don't match between the read and reference then the mapping software soft clips those bases. This can be handled by simply including a few flanking bases in the reference so that additional matches can support the extension of the alignment between read and reference through the region we are interested in.

Once you have extracted corresponding sequence regions from all of the input samples, the next step is to figure out which positions in each sequence correspond. As you have seen while working with the read mapping data over the last week, there can be short insertions and deletions that result in 16S gene sequences being different lengths. That means that you can't simply assume that any given index of two sequences correspond to bases you could meaningfully compare. Instead, you need to identify corresponding positions by aligning the sequences. The Needleman-Wunsch algorithm that you implemented as part of your magnum opus is a pairwise alignment algorithm. That means it can be used to align two sequences to identify corresponding positions.

Once you have aligned two sequences such that corresponding positions in each sequence are lined up you can then identify differences between those sequences. There are different ways you could quantify the differences between two sequences (e.g., are gaps more different than mismatches? is A/T more or less different than A/G?). For our purposes we can keep it simple and just sum the number of aligned positions at which the two sequences differ (i.e., a gap or mismatch at a position is 1 difference). If you align each pair of sequences and quantify difference in that way you will end up with pairwise distances between all the sequences being considered. Pairwise distances are commonly encoded as a matrix where each cell represents the distance between the sequence corresponding to that row and the sequence corresponding to that column (i.e., cell `matrix[a][b]` contains the distance between `sequences[a]` and `sequences[b]`). Because differences between two sequences are the same in either direction (i.e., A vs B is the same as B vs A), the distance matrix is symmetrical about the diagonal (`matrix[a][b] == matrix[b][a]`).

Once you have pairwise distances, you can infer phylogeny using one of the many distance-based phylogeny methods. Distance-based tree inference methods are a way to summarize pairwise relationships as a tree topology. They try to place things that are similar to one another close together in a tree, while placing things that are very different far apart. Because things which are genetically similar to one another tend to be phylogenetically more closely related, a tree that groups things based on genetic distance is also often a good approximation of evolutionary relationships.

For this assignment we will use neighbor-joining. If you would like to you can implement the neighbor-joining algorithm yourself. Otherwise, you can use the [one implemented as part of the biotite package](#). You can see the usage of the biotite `neighbor_joining` function in the "Examples" section of that page. The only difference I would like you to use from that example is to include distances and sample names in your output tree by using `<biotite tree instance>.to_newick(labels=<sequence headers>, round_distance=2)`. Note two things you need to know for this step:

1. biotite expects you to pass a distance matrix as a numpy array. You can see that happening in the example linked above. It's easy to convert `list[list[int]]` into a numpy array as shown in that example.
2. Neighbor-joining produces an unrooted tree. If you haven't worked much with trees before, you may have an expectation about how the output tree will look and how you can interpret it. Unrooted trees are not as simple to interpret as rooted trees. If you are not familiar with unrooted trees then take a look at the [section above where I describe them](#) and ask on ED if you have any questions.

Assignment details

As described above, this week your task is to combine the various components of your magnum_opus into a single package and a script called `magop.py` with a command line interface that does the following:

Given zero or more assembly files and zero or more pairs of reads files, use isPCR, read mapping, and Needleman-Wunsch to extract and align the V4 region of the 16S gene using user-provided primers and reference sequences. Your script should then compare the aligned sequences, calculate pairwise distances among them, and produce a Newick format UPGMA tree.

Some more details about what the above summary means:

- Your script should optionally accept assemblies. If assemblies are specified, your script should accept them as a space separated list. For example `magop.py -a assembly1.fasta assembly2.fasta` or `magop.py -a data/assemblies/*`.
- Your script should optionally accept paired Illumina reads files. As with assemblies, you should accept these as a space separated list. Importantly, you will need to identify which files correspond to a pair. To do so you can assume that the read file names will follow the following regex: `(sampleID)_[12].fastq`. i.e., a sample called "Ecoli" would have read files "Ecoli_1.fastq" and "Ecoli_2.fastq". It is important that you pair the reads so that you map corresponding reads to the references.
- Your script should accept primers as it has in previous weeks (i.e., a path to a fasta file containing the primers). This input should be required
- Your script should accept reference sequences as it did for last week's assignment (i.e., path to a fasta file containing reference sequences). This input should be optional, but you might want to add a check to require it if reads are provided (this would be a sensible input check, but it's not required for the grade)

- If assemblies are provided, you should perform isPCR using the provided primers to extract amplicons. You should only keep one amplicon from each assembly (the 16S gene is often present in multiple copies, but they are usually identical or very similar)
- If reads are provided, you should map each pair of reads to the provided references and then use isPCR to extract one amplicons from each mapping consensus sequence (i.e., for each pair of reads, map and then extract the best consensus, then isPCR to extract an amplicon from that best consensus)
- If references are provided (i.e., when working with reads), you should also use isPCR to extract one amplicon from each sequence in the reference file.
- After extracting all amplicons, you should use Needleman-Wunsch to orient the amplicons consistently and then quantify the distance between each aligned sequence. The distances should then be used to infer a neighbor joining tree.
- Your script should print the inferred neighbor joining tree to the stdout.

The usage for your script should be

```
magop.py [-a ASSEMBLY [ASSEMBLY ...]] -p PRIMERS [-r READS [READS ...]] [-s
REF_SEQS]
```

where square brackets indicate optional inputs and nested square brackets such as `[-a ASSEMBLY [ASSEMBLY ...]]` indicate that one or more input can be provided.

N.B. Important things to note for this assignment:

1. Your output will be assessed by comparing your tree to various expected trees. The autograder will use [this R package](#) to assess whether your tree is similar to the expected tree or a tree that indicates you have made a certain mistake and will give you hints if it can. Importantly, the autograder needs your leaf labels to match those in its comparator trees exactly. Please make sure you are producing trees that match the leaf labels in the below example tree or the autograder will not work.
2. In order for your script to work with the diverse sequences we are using, you will need to modify your isPCR BLAST command to tolerate more mismatches. Specifically, you should add the following to your command: `-word_size 6 -penalty=-2`. Without those settings you won't get any amplicon for the *Spirochaetes* in the dataset.

Expected output tree

I midpoint rooted and rounded the branch lengths of the tree my script produced. You can do the same using by piping your tree to `gotree reroot midpoint | gotree brlen round -p 2`

Below is the expected tree when your script is run on all the provided data.

(((((((((Pseudomonas_syringae_pv_tomato_str_DC3000_NC_004578.1:1.72,Pseudomonas_pro
tegens_CHA0_NZ_LS999205.1:0.28):0.39,Pseudomonas_putida_NBRC_14164_NC_021505.1:0.61
):1.75,(Pseudomonas_oleovorans_GD04132_NZ_CP104579.1:0.89,(SRR21376282:0,
(Pseudomonas_aeruginosa_UCBPP-
PA14_NC_008463.1:0,Pseudomonas_aeruginosa_PA01_NC_002516.2:0):0):6.11):2.13):8.9,
((Vibrio_cholerae_strain_N16961_NZ_CP028827.1:0,ERR13716760:0):10.21,
(Escherichia_coli:0,Escherichia_coli_str._K-
12_substr._MG1655_NC_000913.3:0):9.79):10.31):7.87,
((Wolbachia_NZ_CP046925.1:3.97,Wolbachia_pipientis:9.03):15.77,SRR24105535:17.73):9
.02):3.58,(Mycoplasma_pneumoniae:21.26,SRR25626983:21.74):14.46):1.67,
(((ERR11767307:12.39,Fusibacter_paucivorans:15.61):8.01,
(Leptospira_borgpetersenii:-0.02,SRR13255634:0.02):25.74):2.92,
((SRR30750791:4.01,SRR28858832:5.99):2.38,Bacillus_subtilis:9.12):12.33):2.14):3.61
,(Synechococcus_elongatus:21.69,SRR27732368:23.31):10.53):8.95,
((SRR24886915:10.61,Methanococcus_aeolicus:9.39):24.69,
(Sulfolobus_islandicus:15.04,ERR12954019:12.96):22.81):13.77);